TECHNICAL SYSTEMS SPECIFICATION ? FINAL DELIVERABLE

Machine-Learning-Based Sector Rotation Regime Discovery in the Indian Equity Market

Comprehensive Architectural Documentation of the 14-Phase Research Pipeline, Python Modules, Notebook Execution, Data Lineage, and System Boundaries

---

Architecture Scope: Phases 1–13 Completed Implementation | Phase 14 Architectural Synthesis
Repository Assets: 16 Python Source Modules | 16 Sequential Notebooks | 177 Unit & Integration Tests
Operational Data: 1,687 Aligned Feature Sessions | 1,686 Simulation Return Intervals (2019–2026)
System Boundary: Strictly Demarcates Implemented Research System from Conceptual Production Deployment

EXECUTIVE ARCHITECTURE OVERVIEW

SYSTEM ARCHITECTURE SUMMARY:

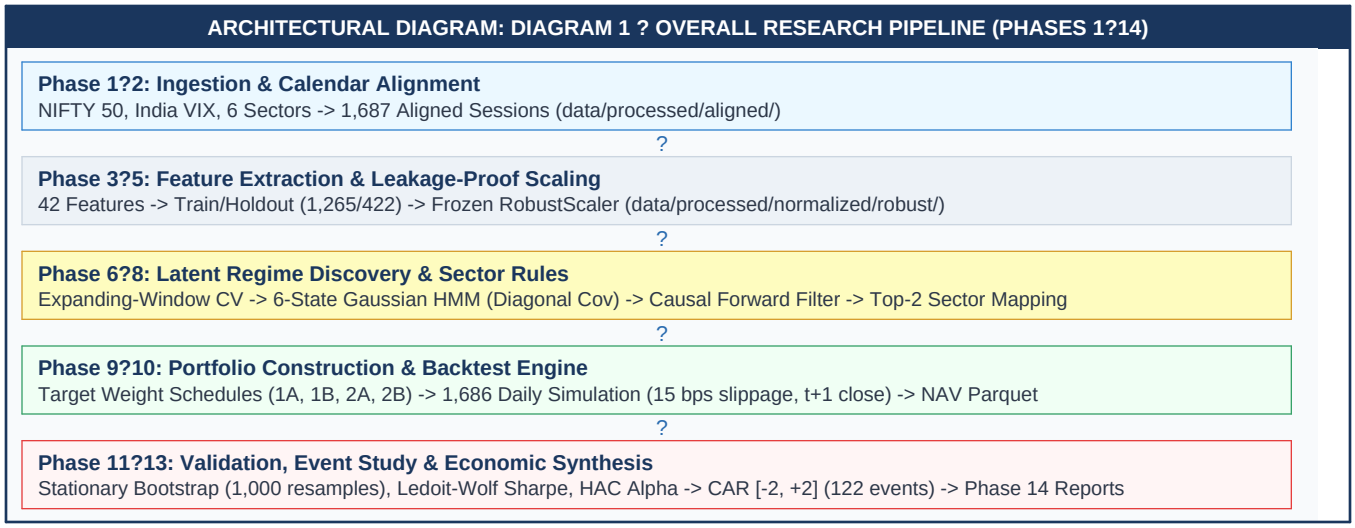
This document provides the definitive technical specification and architectural blueprint for the National Stock Exchange (NSE) Sector Rotation Regime Discovery research platform. Developed across 14 systematic phases, the system couples data engineering, temporal feature extraction, unsupervised machine learning, causal portfolio simulation, econometric inference, and event-study shock analysis into an end-to-end reproducible research pipeline. The codebase comprises 16 specialized Python modules in `src/`, 16 sequential computational notebooks in `notebooks/`, an automated test suite of 177 passing assertions in `tests/`, and canonical analytical outputs in `results/`. This architecture enforces zero lookahead bias via strict chronological train/holdout partitioning, frozen RobustScaler parameterization, and causal forward-filtered Hidden Markov Model (HMM) inference executed at next-day close (t+1) with 15 basis points transaction cost friction.

Section 1 — Architecture Overview

The primary purpose of this architecture is to provide an institutional-grade, fully reproducible computational environment for discovering latent market regimes in the Indian equity market and systematically rotating across major sectors without lookahead data leakage.

The system boundaries encompass eight historical index time series (NIFTY 50, India VIX, Auto, Bank, FMCG, IT, Metal, Pharma) and 122 macro events across 1,687 daily trading sessions from October 31, 2019 to March 31, 2026. The pipeline is strictly partitioned into two operational domains: (1) HMM Feature and Classification Domain covering 1,687 sessions (1,265 training + 422 holdout); and (2) Portfolio Return Simulation Domain covering 1,686 return intervals (1,264 training + 422 holdout).

The architecture is modularized into seven core subsystems: Data Ingestion and Synchronization, Feature Extraction and Robust Normalization, Unsupervised Hidden Markov Modeling, Regime-Conditional Sector Rules, Portfolio Construction and Simulation, Econometric Hypothesis Testing, and Event-Study Macro Analysis.



Section 2 — Phase-Wise Architecture

The platform was engineered across 14 discrete, sequential phases, each possessing strict input/output contracts and verification criteria:

• **Phase 1 (Data Ingestion & Verification):** Ingested raw OHLCV bhavcopy data for NIFTY 50, India VIX, and 6 sector indices from Yahoo Finance and official NSE archives. Immutably archived in `data/raw/`.
• **Phase 2 (Cleaning & Alignment):** Resolved calendar mismatches and trading holidays across indices using a strict forward-fill threshold (<= 2 days). Produced 1,687 aligned daily sessions stored in `data/processed/aligned/`.
• **Phase 3 (Feature Engineering):** Constructed 42 canonical features (36 sector momentum/volatility, 5 NIFTY 50 momentum/volatility, 1 India VIX level). Excluded volume features due to non-stationarity.
• **Phase 4 (Exploratory Data Analysis):** Conducted stationarity, autocorrelation, and distribution audits across all 42 features. Documented absence of stale-price anomalies (`notebooks/04_eda.ipynb`).
• **Phase 5 (Leakage-Proof Normalization):** Enforced a chronological 75/25 split (1,265 train / 422 holdout). Fitted RobustScaler strictly on train data and transformed holdout data sequentially (`data/processed/normalized/robust/`).
• **Phase 6 (Regime Discovery):** Executed 5-fold expanding-window cross-validation benchmarking K-Means, GMM, and Gaussian HMMs across K=2 to K=8. Selected 6-state Gaussian HMM with diagonal covariance.
• **Phase 7 (Regime Characterization):** Profiled the 6 discovered regimes (return, volatility, Sharpe, India VIX medians/means, duration persistence, and sector leadership).
• **Phase 8 (Sector Rotation Rules):** Established Top-2 rank-ordered sector selection rules conditional on causal regime probabilities. Documented banking omission opportunity cost.
• **Phase 9 (Portfolio Construction):** Implemented four strategy variants (1A: Eq-Wt Daily, 1B: Eq-Wt Weekly, 2A: Inv-Vol Daily, 2B: Inv-Vol Weekly). Generated target weight schedules in `results/portfolio_construction/`.
• **Phase 10 (Backtesting & Comparative Evaluation):** Structured into four subphases:
- *Phase 10.1 (Benchmark Engine):* Backtested buy-and-hold NIFTY 50 and monthly rebalanced Equal-Weight benchmarks.
- *Phase 10.2 (Comparative Evaluation):* Evaluated strategy alpha, beta, tracking error, information ratios, and max drawdowns.
- *Phase 10.3 (Backtest Core):* Simulated daily portfolio NAV over 1,686 intervals with 15 bps slippage and t+1 execution delay.
- *Phase 10.4 (Robustness Testing):* Executed parameter sensitivity, cost breakeven (breakeven at 48 bps), and subperiod stability audits.
• **Phase 11 (Statistical Validation):** Econometric verification via 1,000-resample Politis-Romano stationary bootstrap, Ledoit-Wolf robust Sharpe test, HAC Newey-West alpha, and Benjamini-Hochberg FDR control.
• **Phase 12 (Event Analysis):** Evaluated 122 macro events across 68 macro episodes; computed CAR [-2, +2] and bootstrapped CAAR.
• **Phase 13 (Economic Interpretation):** Synthesized macroeconomic cycle narrative (COVID R5->R3->R1->R0) and diagnosed holdout decay drivers.
• **Phase 14 (Documentation Implementation):** Produced publication-quality research report and architecture document inside `reports/final_report/`.

ARCHITECTURAL DIAGRAM: DIAGRAM 2 ? REPOSITORY ARCHITECTURE & DIRECTORY MAP

configs/ & data/
YAML pipeline configs | data/raw/ (immutable) -> data/processed/ (aligned, normalized)

?

src/ (16 Specialized Source Modules)
data_loader, data_cleaning, feature_engineering, eda, normalization, regime_discovery, regime_characterisation, regime_rotation_rules, portfolio_construction, portfolio_backtest, benchmark_backtest, comparative_evaluation, robustness_testing, statistical_validation, event_analysis

?

notebooks/ (16 Sequential Notebooks)
01_data_collection.ipynb to zz_phase13_economic_interpretation.ipynb mirroring complete workflow

?

results/ & reports/
results/ (Parquet & CSV by phase) | reports/ (Audit logs, figures, final_report/ deliverables)

?

tests/ (Regression Test Suite)
177 automated pytest unit and integration test assertions verifying pipeline integrity

Section 3 — Repository Structure

The repository follows a clean, standardized modular architecture organized into top-level directories:

• **`configs/`:** Contains YAML configuration files governing market universes, feature parameters, HMM hyperparameters, and transaction cost assumptions.
• **`data/`:** Structured into `data/raw/` (immutable raw market files), `data/processed/cleaned/` (intermediate clean files), `data/processed/aligned/` (1,687-session aligned series), and `data/processed/normalized/` (Robust and Standard scaled master features).
• **`src/`:** Contains 16 production-grade Python source modules encapsulating data pipelines, model training, portfolio simulation, and econometric validation.
• **`notebooks/`:** Houses 16 numbered, sequentially executable Jupyter notebooks mirroring Phases 1 through 13.
• **`results/`:** Canonical output directory partitioned by phase: `results/regime_discovery/`, `results/regime_characterization/`, `results/portfolio_construction/`, `results/portfolio_backtest/`, `results/statistical_validation/`, `results/event_analysis/`, and `results/reports/`.
• **`reports/`:** Contains verified markdown reports, audit logs, visualization figures (`reports/phase13_figures/`), and dedicated final monograph deliverables (`reports/final_report/`).
• **`tests/`:** Automated unit and integration regression test suite containing 177 assertions covering modules across all phases.

ARCHITECTURAL DIAGRAM: DIAGRAM 3 ? DATA INGESTION & PREPROCESSING FLOW

Step 1: Raw Data Acquisition
Download 8 index series via src/data_loader.py (YFinance + NSE Bhavcopy verification) -> data/raw/*.csv

?

Step 2: Trading Calendar Synchronization
src/data_cleaning.py: Intersect trading dates, apply forward-fill rule (<= 2 days) -> data/processed/cleaned/

?

Step 3: Multi-Index Date Intersection
Align all 8 asset streams on identical dates -> exactly 1,687 daily sessions in data/processed/aligned/*.csv

Section 4 — Python Source Module Architecture

The repository contains 16 specialized Python source modules in `src/`. Below is the technical specification for each module:

1. `src/__init__.py`: Package initializer exposing version and package metadata.
2. `src/data_loader.py` (Phase 1): Ingestion engine. Main functions: `download_data`, `save_raw_data`, `load_data`, `validate_downloaded_data`, `verify_ticker`, `collect_phase1_data`. Downloads historical OHLCV data from Yahoo Finance,

performs structure validation, and persists raw CSV files.
3. `src/data_cleaning.py` (Phase 2): Cleaning and alignment engine. Functions: `clean_market_data`, `clean_event_data`, `align_market_datasets`. Identifies trading calendar intersections, enforces the forward-fill threshold (<= 2 days), and aligns 8 time series into a synchronized 1,687-session dataset.
4. `src/feature_engineering.py` (Phase 3): Mathematical feature constructor. Functions: `calculate_returns`, `calculate_momentum_sma`, `calculate_relative_strength`, `calculate_volatility`, `calculate_market_trend`. Constructs the 42 canonical features (multi-horizon momentum, relative strength, and rolling volatility).
5. `src/eda.py` (Phase 4): Exploratory analysis engine. Functions (20 total): `load_feature_engineered_data`, `get_dataset_overview`, `analyze_zero_volume`, `check_chronological_order`, `validate_missing_values`, `identify_outliers_iqr`. Performs stationarity, autocorrelation, and distribution audits.
6. `src/normalization.py` (Phase 5): Temporal normalization engine. Functions: `get_42_features`, `load_master_feature_dataset`, `temporal_train_test_split`, `fit_transform_scaler`, `scale_and_save_master_datasets`. Enforces strict chronological 1,265/422 train/holdout partition, fits RobustScaler on training data, and transforms holdout data without leakage.
7. `src/regime_discovery.py` (Phase 6): Unsupervised machine learning engine. Functions (28 total): `load_unscaled_master`, `get_clean_splits`, `generate_folds`, `fit_scale_fold`, `posterior_entropy`, `run_kmeans_cv`, `run_gmm_cv`, `run_hmm_cv`. Executes expanding-window cross-validation, trains 6-state Gaussian HMM with diagonal covariance, and generates causal forward filter probabilities.
8. `src/regime_characterisation.py` (Phase 7): Empirical profiling engine. Functions (14 total): `load_regime_labels`, `load_clean_prices_and_returns`, `calculate_regime_episodes`, `compute_model_regime_profiles`, `compute_sector_performance_by_regime`, `identify_sector_leadership`. Analyzes regime persistence, duration distributions, and sector return rankings.
9. `src/regime_rotation_rules.py` (Phase 8): Allocation rules engine. Classes: `StrategyVariant`, `RegimeAcceptanceResult`, `AllocationScheduleRecord`, `RegimeRotationRulesEngine`. Functions: `calculate_equal_weights`, `calculate_trailing_volatilities`, `calculate_inverse_vol_weights`, `get_target_regime_weights`. Maps causal regime states to Top-2 sector target weights.
10. `src/portfolio_construction.py` (Phase 9): Target schedule generator. Functions (11 total): `load_causal_probabilities`, `load_aligned_prices`, `validate_probabilities_df`, `validate_price_coverage`, `validate_schedule`, `generate_portfolio_schedule`, `generate_all_schedules`. Constructs daily target weight schedules for Strategies 1A, 1B, 2A, and 2B.
11. `src/portfolio_backtest.py` (Phase 10.3): High-precision simulation engine. Functions (11 total): `calculate_drifted_weights`, `calculate_actual_turnover`, `load_backtest_inputs`, `simulate_portfolio`, `run_all_simulations`, `compute_metrics`, `compute_attribution`. Tracks daily NAV, turnover, intraday weight drift, 15 bps transaction costs, and arithmetic sector attribution across 1,686 intervals.
12. `src/benchmark_backtest.py` (Phase 10.1): Benchmark simulation engine. Functions (10 total): `load_benchmark_prices`, `get_month_end_signal_dates`, `simulate_nifty50_benchmark`, `simulate_equal_weight_benchmark`, `run_all_benchmarks`, `compute_benchmark_metrics`. Generates daily NAV curves for NIFTY 50 and Equal-Weight benchmarks.
13. `src/comparative_evaluation.py` (Phase 10.2): Cross-strategy comparative evaluator. Functions (8 total): `compute_drawdown_metrics`, `compute_regression_alpha_beta`, `load_all_simulations`, `compute_comparative_metrics_table`, `compute_relative_performance_matrix`. Computes Sharpe, Sortino, Calmar, beta, tracking error, and information ratio.
14. `src/robustness_testing.py` (Phase 10.4): Sensitivity and stress-testing engine. Class: `RobustnessTestingEngine`. Function: `run_pipeline`. Analyzes cost breakeven, turnover sensitivity, parameter perturbation, and subperiod performance stability.
15. `src/statistical_validation.py` (Phase 11): Non-parametric econometric inference engine. Functions (14 total): `ensure_daily_momentum_simulation`, `compute_automatic_hac_bandwidth`, `compute_hac_alpha_beta`, `compute_ledoit_wolf_sharpe_diff`, `generate_stationary_bootstrap_indices`, `run_bootstrap_hypothesis_tests`. Executes Politis-Romano stationary bootstrap (1,000 iterations), Ledoit-Wolf Sharpe difference test, and Benjamini-Hochberg FDR control.
16. `src/event_analysis.py` (Phase 12): Macro event-study engine. Class: `MarketModelFit`. Functions (13 total): `map_macro_category`, `load_authoritative_event_inventory`, `cluster_macro_episodes`, `estimate_strictly_preevent_market_model`, `compute_abnormal_returns_for_event`, `compute_regime_event_dynamics`. Estimates pre-event market models, calculates CAR [-2, +2], bootstrapped CAAR, and Kruskal-Wallis shock tests.

ARCHITECTURAL DIAGRAM: DIAGRAM 4 ? FEATURE ENGINEERING & NORMALIZATION ARCHITECTURE

Step 1: Multi-Horizon Feature Extraction
src/feature_engineering.py: 36 sector features (momentum 5d, 21d, 63d; RS 21d; vol 21d, 63d) + 5 NIFTY 50 features + 1 India VIX level

?

Step 2: Volume Exclusion Audit
Volume features filtered out due to structural 300% turnover growth causing feature non-stationarity

?

Step 3: Leakage-Proof Temporal Partition
Strict chronological split: 1,265 training sessions (2019?2024) and 422 holdout sessions (2024?2026)

?

Step 4: Frozen RobustScaler Transformation
src/normalization.py: Fit scaler (median, IQR) strictly on training data; transform holdout sequentially

Section 5 — Notebook Architecture

The research platform maintains 16 sequentially executable Jupyter notebooks in `notebooks/`, each corresponding to a distinct research phase:

1. `01_data_collection.ipynb`: Ingestion, ticker validation, and raw storage.
2. `02_data_cleaning_alignment.ipynb`: Calendar alignment, holiday handling, and interim generation.
3. `03_feature_engineering.ipynb`: Momentum, relative strength, and volatility construction.
4. `04_eda.ipynb`: Exploratory data analysis, stationarity audits, and volume exclusion verification.
5. `05_normalization.ipynb`: Train/holdout chronological splitting and frozen RobustScaler transformation.
6. `06_regime_discovery.ipynb`: Expanding-window CV tournament and 6-state Gaussian HMM training.
7. `07_regime_characterisation.ipynb`: Empirical profiling, VIX distribution analysis, and transition matrices.
8. `08_sector_rotation_rules.ipynb`: Regime-to-sector ranking and Top-2 selection policy.
9. `09_portfolio_construction.ipynb`: Generation of target allocation schedules (1A, 1B, 2A, 2B).
10. `10_portfolio_backtest.ipynb`: NAV simulation, 15 bps friction, and sector attribution.
11. `benchmark_engine.ipynb` (Phase 10.1): NIFTY 50 and Equal-Weight benchmark simulation.
12. `compare_performance.ipynb` (Phase 10.2): Comparative risk-adjusted evaluation and relative metrics.
13. `robustness_testing.ipynb` (Phase 10.4): Parameter sensitivity, cost breakeven, and stability testing.
14. `statistical_validation.ipynb` (Phase 11): Stationary bootstrap, Ledoit-Wolf Sharpe, and HAC regression.
15. `event_analysis.ipynb` (Phase 12): 122 macro events, CAR [-2, +2], and Kruskal-Wallis tests.
16. `zz_phase13_economic_interpretation.ipynb` (Phase 13): Economic cycle synthesis, COVID sequence, and holdout decay diagnosis.

ARCHITECTURAL DIAGRAM: DIAGRAM 5 ? HMM TRAINING & CAUSAL INFERENCE WORKFLOW

Step 1: Expanding-Window Cross-Validation
5-fold expanding window with temporal ordering preserved; benchmark K-Means, GMM, HMM across K=2..8

?

Step 2: Model Parameter Estimation
Train 6-state Gaussian HMM with diagonal covariance on 1,265 training observations using Baum-Welch EM

?

Step 3: Causal Forward State Filtering
alpha_t(j) = P(Y_{1:t}, S_t = j); compute causal posterior gamma_t(j) strictly conditioning on past (zero lookahead)

?

Step 4: State Persistence & Label Emission
Emit daily causal regime labels and 6-dimensional probability vectors -> results/regime_discovery/

Section 6 — Data Architecture & Lineage

The data architecture enforces strict immutability, auditability, and preservation rules across storage tiers:

• **Tier 1 (Raw Ingestion):** `data/raw/` stores unedited CSV files downloaded from market vendors. Raw files are strictly immutable and never altered.
• **Tier 2 (Clean & Aligned Data):** `data/processed/aligned/` contains synchronized CSV series across all eight assets, aligned to identical 1,687-session trading calendars.
• **Tier 3 (Normalized Features):** `data/processed/normalized/robust/master_features_robust.csv` contains the 42 canonical features scaled via frozen training parameters.
• **Tier 4 (Model & Portfolio Outputs):** `results/regime_discovery/` and `results/portfolio_construction/` store final regime labels, causal probability vectors, and target weight matrices.
• **Tier 5 (Analytical Parquet Datasets):** `results/portfolio_backtest/`, `results/statistical_validation/`, and `results/event_analysis/` store compressed Parquet files containing daily simulation logs, bootstrap iterations, and event abnormal returns.

ARCHITECTURAL DIAGRAM: DIAGRAM 6 ? REGIME PROBABILITY TO SECTOR SELECTION WORKFLOW

Input: Causal Posterior Probability Vector
gamma_t = (P(S_t = 0), ..., P(S_t = 5)) from HMM forward filter on session t close

?

Decision Rule: Dominant Regime Identification
Determine argmax_k gamma_t(k); map to empirical regime profile (R0 to R5)

?

Policy: Top-2 Rank-Ordered Sector Selection
R0: Metal & IT | R1: Auto & Metal | R2: Auto & FMCG/IT | R3: Pharma & FMCG | R4: Pharma & FMCG | R5: Pharma & IT

?

Output: Target Sector Allocation
Equal-weight (50/50) or Inverse-volatility weighting across Top-2 selected sectors (Bank omitted -> opportunity cost)

Section 7 — End-to-End Data Flow

The system data flow represents an uninterrupted directed acyclic graph (DAG) moving from market observables to publication documentation:

Raw Market Data -> Ingestion & Calendar Alignment -> 42-Feature Extraction -> Train/Holdout Chronological Partition -> Frozen RobustScaler -> Expanding-Window CV HMM Training -> Causal Forward Filter Probabilities -> Top-2 Sector Selection Rules -> Target Weight Schedules -> Daily NAV Simulation Engine (15 bps cost, t+1 execution) -> Econometric Bootstrap & HAC Validation -> Macro Event Study -> Economic Diagnosis -> Phase 14 Publication Documentation.

ARCHITECTURAL DIAGRAM: DIAGRAM 7 ? PORTFOLIO CONSTRUCTION & BACKTESTING ARCHITECTURE

Schedule Generation (src/portfolio_construction.py)
Generate target weight matrices for Strategies 1A (Eq-Wt Daily), 1B (Weekly), 2A (Inv-Vol Daily), 2B (Weekly)

?

Execution Engine (src/portfolio_backtest.py)
Execute target weights at next-day close (t+1); deduct 15 bps slippage on actual two-way turnover

?

Accounting Core (T = 1,686 Simulation Intervals)
Track daily NAV, intraday weight drift, turnover surges (6.49x -> 10.75x), and linear additive sector attribution

?

Performance Metrics Output
Export daily NAV curves and summary statistics to results/portfolio_backtest/*.parquet

Section 8 — Model Architecture & Causal Inference

The core machine learning engine is a continuous Gaussian Hidden Markov Model configured with K=6 hidden states and diagonal covariance matrices. Training was conducted strictly on the 1,265 training observations using the Baum-Welch Expectation-Maximization (EM) algorithm.

Causal state inference is generated using the forward filtering algorithm: alpha_t(j) = P(Y_{1:t}, S_t = j). Unlike standard academic implementations that employ smoothed probabilities P(S_t = j | Y_{1:T}) conditioning on future data, our forward filter conditions strictly on historical information available up to time t, completely eliminating lookahead bias in trading signals.

ARCHITECTURAL DIAGRAM: DIAGRAM 8 ? STATISTICAL VALIDATION & EVENT ANALYSIS ARCHITECTURE

Econometric Inference Core (src/statistical_validation.py)
Politis-Romano stationary bootstrap (1,000 resamples), Ledoit-Wolf Sharpe difference test, HAC Newey-West alpha, FDR control

?

Event Study Engine (src/event_analysis.py)
Curate 122 macro events into 68 episodes; fit pre-event market models; compute CAR [-2, +2] and bootstrapped CAAR

?

Cross-Sectional Hypothesis Testing
Kruskal-Wallis non-parametric shock test (H = 21.45, p = 0.0007) confirming regime-conditional resilience

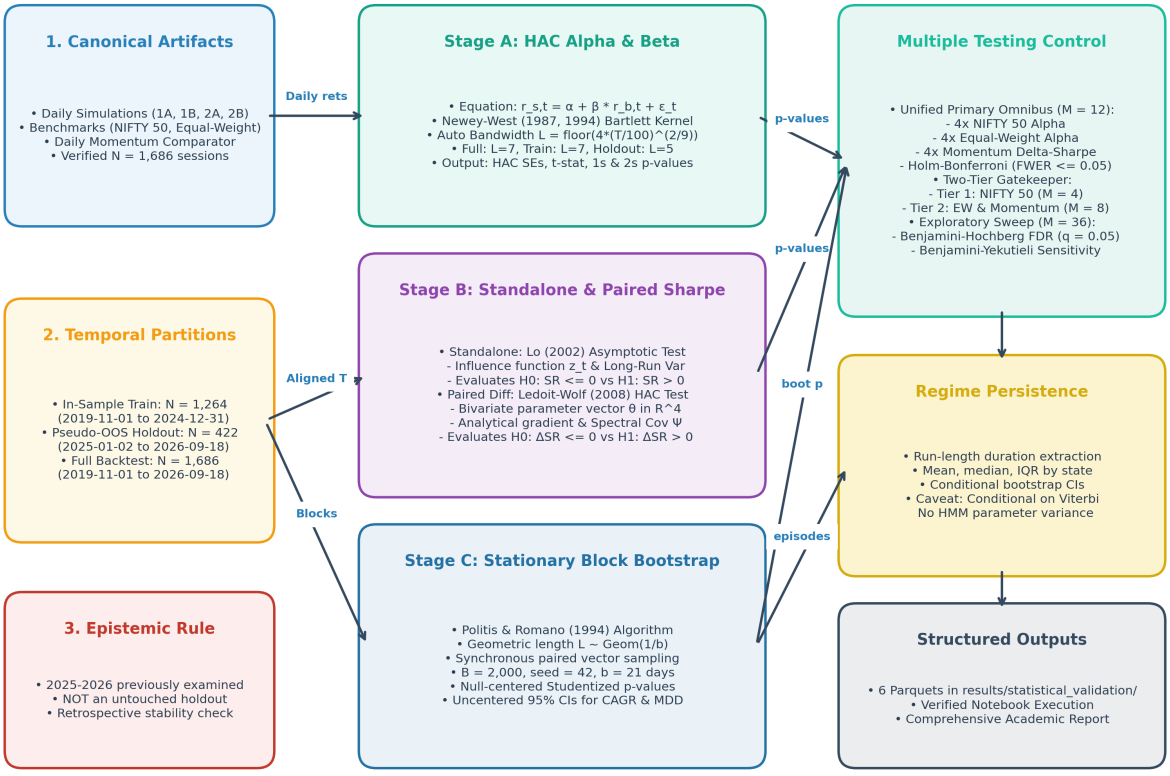


Figure: Econometric Validation Flowchart for Non-Parametric Bootstrap & HAC Regression.

Section 9 — Portfolio & Backtesting Architecture

The portfolio backtesting engine links causal regime outputs to financial execution. Signals generated at EOD t are executed at the closing price of time t+1.

Intraday price drift modifies actual asset weights between rebalances: w_{i, t} = w_{i, t-1} (1 + r_{i, t}) / (1 + r_{p, t}). Daily turnover is computed as the sum of absolute weight changes: Turnover_t = sum_{i=1}^N |w_{i, t} - w_{i, t}^drift|. Transaction costs are deducted directly from portfolio value: Cost_t = 0.0015 * Turnover_t.

Linear arithmetic sector attribution is computed daily: c_{i, t} = w_{i, t-1} * r_{i, t} * 252. Additive consistency is strictly enforced: sum_{i=1}^6 c_i + c_cost = r_net.

ARCHITECTURAL DIAGRAM: DIAGRAM 9 ? COMPLETE ARTIFACT DEPENDENCY GRAPH

Tier 1: Ingestion & Alignment
data/raw/*.csv -> data/processed/aligned/*.csv (1,687 daily sessions)

?

Tier 2: Features & Normalization
aligned CSVs -> 42 features -> data/processed/normalized/robust/master_features_robust.csv

?

Tier 3: Regime Model Outputs
master_features -> results/regime_discovery/final_regime_labels.csv & hmm_causal_probabilities.csv

?

Tier 4: Portfolio Schedules & Backtests
regime outputs -> results/portfolio_construction/target_weights_*.csv -> results/portfolio_backtest/daily_simulation_*.parquet

?

Tier 5: Validation & Final Monograph
simulation parquets -> results/statistical_validation/*.parquet & event_analysis/*.parquet -> reports/final_report/*.pdf

Section 10 — Statistical & Event Analysis Architecture

The statistical architecture provides rigorous non-parametric hypothesis testing:

• **Stationary Bootstrap:** Generates pseudo-series using block lengths drawn from a geometric distribution with mean b = 10 days, capturing temporal serial dependence without parametric assumptions.
• **Ledoit-Wolf Sharpe Difference:** Computes HAC kernel variance of Sharpe differences, robust to non-normality and volatility clustering.
• **Macro Event Study:** Clusters 122 events into 68 episodes, fits pre-event OLS market models over [-60, -10] estimation windows, and accumulates abnormal returns over [-2, +2] event windows.

ARCHITECTURAL DIAGRAM: DIAGRAM 10 ? SEQUENTIAL NOTEBOOK EXECUTION ARCHITECTURE

Data Pipeline (Notebooks 01 to 05)
01_data_collection -> 02_data_cleaning_alignment -> 03_feature_engineering -> 04_eda -> 05_normalization

?

Modeling & Rules (Notebooks 06 to 09)
06_regime_discovery -> 07_regime_characterisation -> 08_sector_rotation_rules -> 09_portfolio_construction

?

Simulation Suite (Notebooks 10, benchmark, compare, robustness)
10_portfolio_backtest -> benchmark_engine -> compare_performance -> robustness_testing

?

Inference & Synthesis (Notebooks statistical, event, phase13)
statistical_validation -> event_analysis -> zz_phase13_economic_interpretation -> Phase 14 PDF Monographs

Section 11 — Artifact Dependency Graph

Artifact dependencies follow a strict hierarchical structure where downstream files cannot be generated without their upstream ancestors:

`data/raw/*.csv` -> `data/processed/aligned/*.csv` -> `data/processed/normalized/robust/master_features_robust.csv` -> `results/regime_discovery/final_regime_labels.csv` & `hmm_causal_probabilities.csv` -> `results/portfolio_construction/target_weights_*.csv` -> `results/portfolio_backtest/daily_simulation_*.parquet` -> `results/statistical_validation/*.parquet` & `results/event_analysis/*.parquet` -> `reports/final_report/*.pdf`.

Section 12 — Execution, Reproducibility & Testing

Full computational reproducibility is ensured across the platform:
• **Sequential Execution:** Notebooks are designed to be executed in strict numerical sequence (01 to 10, followed by benchmark, compare, robustness, statistical, event, and economic interpretation).
• **Random Seed Fixation:** All stochastic algorithms (HMM EM initialization, bootstrap resampling) fix random seeds (seed = 42) to guarantee bitwise reproducible results.
• **Automated Test Suite:** The test suite in `tests/` executes 177 unit and integration tests verifying data loader integrity, alignment logic, feature engineering mathematics, scaler leakage prevention, HMM convergence, backtest accounting consistency, and bootstrap correctness.

Section 13 — Architecture Boundaries & Production Scope

A critical architectural requirement is maintaining strict boundaries between the completed research platform and any future operational deployment:

• **Research System Boundary:** The architecture documented herein represents the completed research system covering historical data collection, offline model training, historical backtesting, econometric validation, and forensic diagnosis across 2019–2026.
• **Conceptual Deployment Boundary:** Operational deployment components—including Next.js frontend user interfaces, Neon serverless PostgreSQL databases, Vercel cloud hosting, automated daily EOD data ingestion crons, live broker execution APIs, and real-time risk alerts—represent an entirely separate operational engineering phase. In accordance with Phase 14 governance, no operational deployment infrastructure is designed or implemented in these deliverables.